MedMerge — source code

Repo: github.com/shahriarsharif406/medmerge. data/medmerge-data.json (1.6 MB, generated by scripts/build_data.py) and data/samples.json omitted from this listing.

index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>MedMerge — one safe medicine list from every prescription</title>
<meta name="description" content="MedMerge merges prescriptions from different doctors into one list and flags double doses hiding behind different Bangladeshi brand names.">
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Fraunces:opsz,wght@9..144,500;9..144,700&family=Hind+Siliguri:wght@400;500;600&family=IBM+Plex+Mono:wght@400;500&family=Instrument+Sans:wght@400;500;600&display=swap" rel="stylesheet">
<link rel="icon" href="favicon.svg" type="image/svg+xml">
<link rel="stylesheet" href="style.css">
</head>
<body>
<header class="top">
  <a class="brand" href="#" aria-label="MedMerge">
    <svg viewBox="0 0 40 40" width="34" height="34" aria-hidden="true">
      <rect x="2" y="2" width="36" height="36" rx="9" fill="#14231D"/>
      <circle cx="13" cy="14" r="4.2" fill="#F6F1E7"/><circle cx="27" cy="14" r="4.2" fill="#F6F1E7"/>
      <circle cx="13" cy="26" r="4.2" fill="#F6F1E7"/><circle cx="27" cy="26" r="4.2" fill="#D9482B"/>
    </svg>
    <span>MedMerge</span>
  </a>
  <nav>
    <button class="ghost" id="keyBtn" data-i18n="settings">Scan settings</button>
    <div class="lang" role="group" aria-label="Language">
      <button data-lang="en" class="on">EN</button><button data-lang="bn">বাংলা</button>
    </div>
  </nav>
</header>

<main>
  <section class="hero">
    <p class="eyebrow" data-i18n="eyebrow">AI × Healthcare · Medication safety</p>
    <h1 data-i18n="h1">Three doctors.<br><em>One safe list.</em></h1>
    <p class="lede" data-i18n="lede">In Bangladesh the same medicine sells under dozens of brand names. When prescriptions from different doctors pile up, a double dose can hide behind two different names. MedMerge reads them all, merges them, and shows what needs a second look.</p>
    <div class="samples" id="samples"></div>
  </section>

  <section class="bench" id="bench" aria-live="polite">
    <div class="bench-head">
      <h2 data-i18n="step1">1 · Your prescriptions</h2>
      <div class="actions">
        <label class="btn" for="photo"><span data-i18n="scan">Scan a photo</span></label>
        <input type="file" id="photo" accept="image/*" hidden>
        <button class="btn" id="addRx" data-i18n="addRx">+ Add prescription</button>
        <button class="btn ghost" id="clearAll" data-i18n="clear">Clear</button>
      </div>
    </div>
    <p class="hint" data-i18n="hint">One medicine per line. Brand or generic name, strength optional. Example: <code>Tab. Napa 500 mg 1+1+1</code></p>
    <div class="rx-lane" id="lane"></div>
    <p class="status" id="scanStatus" role="status"></p>
  </section>

  <section class="result" id="result" hidden>
```

```
    <div class="verdict" id="verdict"></div>
    <div class="cols">
      <div>
        <h2 data-i18n="step2">2 · What needs a second look</h2>
        <div id="findings"></div>
      </div>
      <div>
        <h2 data-i18n="step3">3 · Your merged list</h2>
        <div id="merged"></div>
        <div id="unknown"></div>
        <div class="doc-q" id="docq"></div>
        <button class="btn solid" id="print" data-i18n="print">Save / print this list</button>
      </div>
    </div>
  </section>
</main>

<footer>
  <p data-i18n="disclaimer"><strong>Not medical advice.</strong> MedMerge is a decision-support prototype buil
t for a hackathon. Never stop or change a medicine because of this page — show the list to your doctor or phar
macist. In an emergency call 999.</p>
  <p class="small" id="credit">Brand and label data: <a href="https://github.com/wsayan/medicinedb">wsayan/med
icinedb</a> (MIT), compiled from public Bangladeshi drug-index data. Interaction wording comes from those labe
l summaries and has not been clinically validated.</p>
</footer>

<dialog id="keyDlg">
  <form method="dialog">
    <h3 data-i18n="keyTitle">Scan settings</h3>
    <p data-i18n="keyText">Photo scanning uses Google Gemini from your browser. Paste your own API key; it sta
ys on this device and is sent only to Google. No key? Type the names or use a sample.</p>
    <input id="keyInput" type="password" placeholder="Gemini API key" autocomplete="off">
    <menu><button value="cancel" class="btn ghost" data-i18n="cancel">Cancel</button><button value="save" clas
s="btn solid" id="saveKey" data-i18n="save">Save</button></menu>
  </form>
</dialog>
<script type="module" src="app.js"></script>
</body>
</html>
```

style.css

```
:root{
  --paper:#F6F1E7; --card:#FFFDF8; --ink:#14231D; --ink2:#4A5A52; --green:#1F5C4A; --mint:#DCE8DF;
  --coral:#D9482B; --coral-bg:#FBE6DF; --amber:#B97A0A; --amber-bg:#F8EBCB; --line:#DDD3BF; --ok:#2E7D5B;
  --serif:'Fraunces',Georgia,serif; --sans:'Instrument Sans','Hind Siliguri',system-ui,sans-serif; --mono:'IBM
Plex Mono',ui-monospace,monospace;
}
*{box-sizing:border-box}
html{scroll-behavior:smooth}
body{margin:0;background:var(--paper);color:var(--ink);font:16px/1.55 var(--sans);
  background-image:radial-gradient(circle at 12% -10%,#E9E0CB 0,transparent 40%),radial-gradient(circle at 10
0% 0,#DCE8DF 0,transparent 35%)}
body.bn{font-family:'Hind Siliguri','Instrument Sans',sans-serif}
.top{display:flex;justify-content:space-between;align-items:center;padding:18px clamp(16px,4vw,48px);max-widt
h:1240px;margin:auto}
.brand{display:flex;align-items:center;gap:10px;text-decoration:none;color:var(--ink);font:700 22px var(--seri
f);letter-spacing:-.02em}
.top nav{display:flex;gap:12px;align-items:center}
.lang{display:flex;border:1px solid var(--ink);border-radius:999px;overflow:hidden}
.lang button{border:0;background:none;padding:6px 12px;font:500 13px var(--sans);cursor:pointer;color:var(--in
k)}
.lang button.on{background:var(--ink);color:var(--paper)}
main{max-width:1240px;margin:auto;padding:0 clamp(16px,4vw,48px) 40px}
.hero{padding:clamp(24px,6vw,72px) 0 28px;max-width:860px}
.eyebrow{font:500 12px var(--mono);letter-spacing:.14em;text-transform:uppercase;color:var(--green);margin:0 0
14px}
h1{font:700 clamp(40px,7.5vw,84px)/.98 var(--serif);letter-spacing:-.035em;margin:0 0 20px}
h1 em{font-style:italic;color:var(--coral);font-weight:500}
body.bn h1{font-family:'Hind Siliguri',serif;line-height:1.15;letter-spacing:0}
```

```
.lede{font-size:clamp(16px,2vw,19px);color:var(--ink2);max-width:62ch;margin:0 0 26px}
.samples{display:grid;grid-template-columns:repeat(auto-fit,minmax(230px,1fr));gap:12px}
.sample{all:unset;cursor:pointer;background:var(--card);border:1px solid var(--line);border-radius:16px;paddin
g:14px 16px;display:block;transition:.18s}
.sample:hover,.sample:focus-visible{transform:translateY(-3px);border-color:var(--ink);box-shadow:0 10px 24px
-14px #14231D88}
.sample b{display:block;font:600 15px/1.3 var(--sans)}
.sample span{display:block;font-size:12.5px;color:var(--ink2);margin-top:4px}
.sample i{font:500 11px var(--mono);color:var(--green);letter-spacing:.08em;text-transform:uppercase;font-styl
e:normal}

h2{font:600 15px var(--mono);letter-spacing:.06em;text-transform:uppercase;margin:0 0 14px;color:var(--ink)}
.bench{background:var(--ink);color:var(--paper);border-radius:28px;padding:clamp(18px,3vw,32px);margin-top:20p
x}
.bench h2{color:var(--paper)}
.bench-head{display:flex;justify-content:space-between;gap:12px;flex-wrap:wrap;align-items:center}
.actions{display:flex;gap:8px;flex-wrap:wrap}
.btn{font:500 14px var(--sans);border:1px solid currentColor;border-radius:999px;padding:8px 16px;cursor:point
er;background:none;color:inherit;display:inline-block}
.bench .btn{color:var(--paper)}
.btn:hover{background:#ffffff1f}
.btn.ghost{border-color:transparent;opacity:.8}
.top .btn.ghost{color:var(--ink)}
.btn.solid{background:var(--ink);color:var(--paper);border-color:var(--ink)}
.btn.solid:hover{background:var(--green)}
.hint{font-size:13.5px;color:#C9D4CC;margin:10px 0 16px}
code{font:13px var(--mono);background:#ffffff1c;padding:1px 6px;border-radius:5px}
.rx-lane{display:flex;gap:16px;overflow-x:auto;padding:4px 2px 12px;scroll-snap-type:x proximity}
.rx{flex:0 0 min(330px,86vw);background:var(--card);color:var(--ink);border-radius:18px;padding:16px;scroll-sn
ap-align:start;position:relative;
  background-image:repeating-linear-gradient(transparent 0 27px,#E9E1CF 27px 28px);background-position:0 70px;
background-repeat:no-repeat}
.rx header{display:flex;justify-content:space-between;align-items:baseline;gap:8px;border-bottom:2px solid var
(--ink);padding-bottom:8px;margin-bottom:8px}
.rx .rxn{white-space:nowrap;font:500 11px var(--mono);letter-spacing:.1em;color:var(--coral)}
.rx input.doc{border:0;background:none;font:600 15px var(--sans);width:100%;color:var(--ink);padding:0}
.rx input.doc:focus{outline:2px solid var(--mint)}
.rx textarea{width:100%;min-height:112px;border:0;background:transparent;resize:vertical;font:500 14px/28px va
r(--mono);color:var(--ink);padding:0}
.rx textarea:focus{outline:2px solid var(--mint);border-radius:4px}
.rx .rm{position:absolute;top:8px;right:10px;border:0;background:none;cursor:pointer;color:var(--ink2);font-si
ze:18px}
.lines{list-style:none;margin:10px 0 0;padding:10px 0 0;border-top:1px dashed var(--line);display:grid;gap:6p
x;font-size:13px}
.lines li{display:flex;gap:8px;align-items:center;flex-wrap:wrap}
.tag{font:500 11px var(--mono);padding:2px 8px;border-radius:999px;white-space:nowrap}
.tag.ok{background:var(--mint);color:var(--green)}.tag.fz{background:var(--amber-bg);color:var(--amber)}.tag.n
o{background:var(--coral-bg);color:var(--coral)}
.lines .arrow{color:var(--ink2)}
.status{min-height:1.4em;font-size:13.5px;color:#C9D4CC;margin:6px 0 0}

.result{margin-top:34px}
.verdict{border-radius:24px;padding:clamp(18px,3vw,30px);display:flex;gap:22px;align-items:center;flex-wrap:wr
ap;margin-bottom:26px}
.verdict.high{background:var(--coral-bg);border:2px solid var(--coral)}
.verdict.medium{background:var(--amber-bg);border:2px solid var(--amber)}
.verdict.ok{background:var(--mint);border:2px solid var(--ok)}
.verdict .big{font:700 clamp(54px,9vw,96px)/.9 var(--serif);letter-spacing:-.04em}
.verdict.high .big{color:var(--coral)}.verdict.medium .big{color:var(--amber)}.verdict.ok .big{color:var(--o
k)}
.verdict h3{font:700 clamp(20px,3vw,28px)/1.15 var(--serif);margin:0 0 6px}
body.bn .verdict h3{font-family:'Hind Siliguri',serif}
.verdict p{margin:0;color:var(--ink2);max-width:56ch}
.cols{display:grid;grid-template-columns:1.15fr 1fr;gap:clamp(18px,3vw,40px)}
@media(max-width:900px){.cols{grid-template-columns:1fr}}
.f{background:var(--card);border:1px solid var(--line);border-left:6px solid var(--amber);border-radius:14px;p
adding:14px 16px;margin-bottom:12px;animation:rise .35s both}
.f.high{border-left-color:var(--coral)}
@keyframes rise{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:none}}
```

```
.f .t{display:flex;gap:8px;align-items:center;flex-wrap:wrap;margin-bottom:6px}
.f .t b{font:700 17px var(--serif)}
body.bn .f .t b{font-family:'Hind Siliguri',serif}
.pill{font:500 11px var(--mono);letter-spacing:.06em;text-transform:uppercase;padding:2px 9px;border-radius:99
9px;background:var(--coral-bg);color:var(--coral)}
.pill.m{background:var(--amber-bg);color:var(--amber)}
.chips{display:flex;gap:6px;flex-wrap:wrap;margin:6px 0}
.chip{font:500 13px var(--mono);background:var(--ink);color:var(--paper);padding:3px 10px;border-radius:8px}
.f p{margin:4px 0;font-size:14.5px}
.f q{display:block;font-size:13px;color:var(--ink2);border-left:3px solid var(--line);padding-left:10px;margi
n:8px 0 0;quotes:none}
.m-row{display:grid;grid-template-columns:1fr auto;gap:4px 12px;background:var(--card);border:1px solid var(--
line);border-radius:12px;padding:10px 14px;margin-bottom:8px}
.m-row.flag{border-color:var(--coral);background:#FFF6F2}
.m-row b{font:600 15px var(--sans)}
.m-row small{color:var(--ink2);font:400 12.5px var(--mono)}
.m-row .brands{grid-column:1/-1;display:flex;gap:6px;flex-wrap:wrap}
.m-row .brands span{font:500 12.5px var(--mono);background:var(--mint);color:var(--green);padding:2px 8px;bord
er-radius:6px}
.m-row.flag .brands span{background:var(--coral-bg);color:var(--coral)}
.doc-q{background:var(--mint);border-radius:14px;padding:14px 18px;margin:18px 0}
.doc-q h4{margin:0 0 6px;font:600 13px var(--mono);letter-spacing:.06em;text-transform:uppercase;color:var(--g
reen)}
.doc-q ul{margin:0;padding-left:18px;font-size:14.5px}
.unk{background:var(--amber-bg);border-radius:12px;padding:10px 14px;margin-top:10px;font-size:14px}
footer{max-width:1240px;margin:20px auto 40px;padding:0 clamp(16px,4vw,48px);color:var(--ink2);font-size:13.5p
x}
footer a{color:var(--green)}
.small{font-size:12px}
dialog{border:0;border-radius:18px;padding:24px;max-width:460px;width:92vw;background:var(--card);color:var(--
ink)}
dialog::backdrop{background:#14231D99}
dialog input{width:100%;padding:10px 12px;border:1px solid var(--line);border-radius:10px;font:14px var(--mon
o);margin:8px 0}
dialog menu{display:flex;justify-content:flex-end;gap:8px;padding:0;margin:8px 0 0}
dialog .btn.ghost{color:var(--ink)}
:focus-visible{outline:3px solid var(--coral);outline-offset:2px}
@media print{
  body{background:#fff}.top,.hero,.bench,#print,footer .small{display:none!important}
  .result{margin:0}.f,.m-row{break-inside:avoid}
  .verdict{border-width:1px}
}
@media(prefers-reduced-motion:reduce){*{animation:none!important;transition:none!important}}
```

app.js

```
import { buildIndex, reconcile, resolveItem, titleCase } from './engine.js';

const $ = (s) => document.querySelector(s);
const esc = (s) => String(s ?? '').replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"':
'&quot;', "'": '&#39;' }[c]));

const T = {
  en: {
    settings: 'Scan settings', eyebrow: 'AI × Healthcare · Medication safety',
    h1: 'Three doctors.<br><em>One safe list.</em>',
    lede: 'In Bangladesh the same medicine sells under dozens of brand names. When prescriptions from differen
t doctors pile up, a double dose can hide behind two different names. MedMerge reads them all, merges them, an
d shows what needs a second look.',
    step1: '1 · Your prescriptions', scan: 'Scan a photo', addRx: '+ Add prescription', clear: 'Clear',
    hint: 'One medicine per line. Brand or generic name, strength optional. Example: <code>Tab. Napa 500 mg 1+
1+1</code>',
    step2: '2 · What needs a second look', step3: '3 · Your merged list', print: 'Save / print this list',
    disclaimer: '<strong>Not medical advice.</strong> MedMerge is a decision-support prototype built for a hac
kathon. Never stop or change a medicine because of this page — show the list to your doctor or pharmacist. In
an emergency call 999.',
    keyTitle: 'Scan settings', keyText: 'Photo scanning uses Google Gemini from your browser. Paste your own A
PI key; it stays on this device and is sent only to Google. No key? Type the names or use a sample.',
    cancel: 'Cancel', save: 'Save', trySample: 'Try sample', doctor: 'Doctor / clinic', rx: 'Rx',
    meds: 'medicines', distinct: 'distinct ingredients', remove: 'Remove',
```

```
    verdictHigh: (n) => [`${n}`, n === 1 ? 'overlap to check before the next dose' : 'overlaps to check before the next dose', 'Some medicines on these prescriptions overlap. Show this list to your doctor or pharmacist before taking more. Do not stop anything on your own.'],
    verdictMed: (n) => [`${n}`, 'point to ask your doctor about', 'Nothing looks like a double dose, but a few combinations are worth a question at the next visit.'],
    verdictOk: () => ['✓', 'No overlaps found', 'No duplicate ingredients or known interactions were found among the medicines MedMerge recognised. This is not a guarantee of safety.'],
    verdictEmpty: ['—', 'Add at least two medicines', 'MedMerge compares medicines across prescriptions.'],
    sevSerious: 'Serious per label', sevCaution: 'Caution', checkNow: 'Check first', askDoc: 'Ask your doctor',
    SAME_GENERIC: ['Same medicine, different names', (l, g) => `${l} are all <b>${g}</b>. Taking more than one can mean a double dose.`],
    OVERLAP: ['Hidden overlap', (l, g) => `${l} all contain <b>${g}</b>. One of them is a combination that already includes it.`],
    CLASS_DUP: ['Same family, taken together', (l, g) => `${l} are all ${g}. Doctors rarely intend more than one at once.`],
    SAME_BRAND: ['Repeated across prescriptions', (l, g) => `${l} (${g}) appears on more than one prescription. Make sure it is taken only once.`],
    INTERACTION: ['Possible interaction', (a, b) => `${a} and ${b} may affect each other.`],
    and: ' and ', more: (m) => `Plus ${m} more to ask your doctor about.`,
    source: 'From the drug label summary:',
    groups: { NSAID: 'painkillers of the NSAID type', PPI: 'acid-reducing PPIs', H2: 'H2 acid reducers', STATIN: 'statins', ACEI: 'ACE inhibitors', ARB: 'ARB blood-pressure medicines', CCB: 'calcium-channel blockers', SSRI: 'SSRI antidepressants', BZD: 'benzodiazepines', OPIOID: 'opioids', SU: 'sulfonylurea diabetes medicines', ANTIHIST: 'antihistamines', FLUOROQ: 'quinolone antibiotics', MACROLIDE: 'macrolide antibiotics', TRICYC: 'tricyclic antidepressants' },
    colIngredient: 'Active ingredient', usedAs: 'Sold to you as', fix: 'Did you mean', unknownT: 'Not recognised — type the generic name or check the spelling:',
    dq: 'Questions to ask your doctor',
    q: {
      SAME_GENERIC: (l) => `Do I need all of ${l}, or only one?`,
      OVERLAP: (l) => `${l} overlap — which should I keep?`,
      CLASS_DUP: (l) => `Is it intended that I take ${l} together?`,
      SAME_BRAND: (l) => `${l} is on two prescriptions — is one a repeat?`,
      INTERACTION: (a, b) => `Is it safe to take ${a} with ${b}?`,
    },
    scanBusy: 'Reading the prescription…', scanNoKey: 'Add your Gemini key in “Scan settings” to scan photos — or type the names / use a sample.',
    scanDone: (n) => `Read ${n} medicines from the photo. Please check them below.`, scanFail: 'Could not read that photo. Try a sharper image or type the names.',
    loading: 'Loading the Bangladeshi drug index…',
    sampleTag: 'Sample', scanned: 'Scanned prescription',
  },
  bn: {
    settings: 'স্ক্যান সেটিংস', eyebrow: 'AI × স্বাস্থ্যসেবা · ওষুধের নিরাপত্তা',
    h1: 'তিনজন ডাক্তার।<br><em>একটাই নিরাপদ তালিকা।</em>',
    lede: 'বাংলাদেশে একই ওষুধ কয়েক ডজন ব্র্যান্ড নামে বিক্রি হয়। আলাদা ডাক্তারের প্রেসক্রিপশন জমলে দুটি ভিন্ন নামের আড়ালে একই ওষুধ দুবার চলে যেতে পারে। MedMerge সব প্রেসক্রিপশন পড়ে এক তালিকায় আনে এবং কোথায় আরেকবার দেখা দরকার তা দেখায়।',
    step1: '১ · আপনার প্রেসক্রিপশন', scan: 'ছবি স্ক্যান করুন', addRx: '+ প্রেসক্রিপশন যোগ', clear: 'মুছুন',
    hint: 'প্রতি লাইনে একটি ওষুধ। ব্র্যান্ড বা জেনেরিক নাম, শক্তি না দিলেও চলবে। যেমন: <code>Tab. Napa 500 mg 1+1+1</code>',
    step2: '২ · যা আরেকবার দেখা দরকার', step3: '৩ · আপনার একত্রিত তালিকা', print: 'তালিকা সেভ / প্রিন্ট করুন',
    disclaimer: '<strong>এটি চিকিৎসা পরামর্শ নয়।</strong> MedMerge হাকাথনের জন্য বানানো একটি সহায়ক প্রোটোটাইপ। এই পেজ দেখে কোনো ওষুধ বন্ধ বা বদলাবেন না — তালিকাটি ডাক্তার বা ফার্মাসিস্টকে দেখান। জরুরি অবস্থায় ৯৯৯-এ কল করুন।',
    keyTitle: 'স্ক্যান সেটিংস', keyText: 'ছবি স্ক্যানে আপনার ব্রাউজার থেকে Google Gemini ব্যবহার হয়। নিজের API key দিন; এটি শুধু এই ডিভাইসে থাকে এবং কেবল Google-এ যায়। key না থাকলে নাম টাইপ করুন বা নমুনা ব্যবহার করুন।',
    cancel: 'বাতিল', save: 'সেভ', trySample: 'নমুনা দেখুন', doctor: 'ডাক্তার / ক্লিনিক', rx: 'প্রেসক্রিপশন',
    meds: 'টি ওষুধ', distinct: 'টি আলাদা উপাদান', remove: 'সরান',
    verdictHigh: (n) => [`${n}`, 'মিল পরবর্তী ডোজের আগে দেখা দরকার', 'কিছু ওষুধ একটির সাথে আরেকটি মিলে যাচ্ছে। আরও খাওয়ার আগে তালিকাটি ডাক্তার বা ফার্মাসিস্টকে দেখান। নিজে থেকে কিছু বন্ধ করবেন না।'],
    verdictMed: (n) => [`${n}`, 'বিষয়ে ডাক্তারকে জিজ্ঞেস করুন', 'একই ওষুধ দুবার চলছে এমন কিছু পাওয়া যায়নি, তবে কয়েকটি সংমিশ্রণ নিয়ে পরের ভিজিটে প্রশ্ন করা ভালো।'],
    verdictOk: () => ['✓', 'কোনো মিল বা ঝুঁকি পাওয়া যায়নি', 'MedMerge যেসব ওষুধ চিনতে পেরেছে তাতে একই উপাদান দুবার বা পরিচিত ঝুঁকিপূর্ণ মিথস্ক্রিয়া পাওয়া যায়নি। এটি নিরাপত্তার নিশ্চয়তা নয়।'],
    verdictEmpty: ['—', 'কমপক্ষে দুটি ওষুধ দিন', 'MedMerge প্রেসক্রিপশনগুলোর ওষুধ নিজেদের মধ্যে তুলনা করে।'],
    sevSerious: 'লেবেলে গুরুতর', sevCaution: 'সতর্কতা', checkNow: 'আগে দেখুন', askDoc: 'ডাক্তারকে জিজ্ঞেস করুন',
    SAME_GENERIC: ['একই ওষুধ, ভিন্ন নাম', (l, g) => `${l} — সবই <b>${g}</b>। একাধিক খেলে দ্বিগুণ ডোজ হয়ে যেতে পারে।`],
    OVERLAP: ['লুকানো মিল', (l, g) => `${l} — সবগুলোতেই <b>${g}</b> আছে। একটি কম্বিনেশন ওষুধ, যার ভেতরে ওটি আগে থেকেই আছে।`],
    CLASS_DUP: ['একই গোত্রের ওষুধ একসাথে', (l, g) => `${l} — সবই একই গোত্রের (${g})। ডাক্তাররা সাধারণত একাধিক একসাথে চান না।`],
    SAME_BRAND: ['একাধিক প্রেসক্রিপশনে একই ওষুধ', (l, g) => `${l} (${g}) একাধিক প্রেসক্রিপশনে আছে। নিশ্চিত করুন এটি একবারই খাচ্ছেন।`],
```

```
    INTERACTION: ['সম্ভাব্য মিথস্ক্রিয়া', (a, b) => `${a} ও ${b} একে অপরের কাজে প্রভাব ফেলতে পারে।`],
    and: ' ও ', more: (m) => `এর বাইরে আরও ${m}টি বিষয় ডাক্তারকে জিজ্ঞেস করার আছে।`,
    source: 'ওষুধের লেবেল সারসংক্ষেপ থেকে (ইংরেজিতে):',
    groups: { NSAID: 'NSAID ধরনের ব্যথানাশক', PPI: 'গ্যাস্ট্রিকের PPI', H2: 'H2 গ্যাস্ট্রিক ওষুধ', STATIN: 'স্ট্যাটিন', ACEI: 'ACE ইনহিবিটর',
ARB: 'ARB রক্তচাপের ওষুধ', CCB: 'ক্যালসিয়াম-চ্যানেল ব্লকার', SSRI: 'SSRI অবসাদের ওষুধ', BZD: 'বেঞ্জোডায়াজেপিন', OPIOID: 'ওপিওয়েড', SU: 'সাল
ফোনাইলইউরিয়া ডায়াবেটিসের ওষুধ', ANTIHIST: 'অ্যান্টিহিস্টামিন', FLUOROQ: 'কুইনোলোন অ্যান্টিবায়োটিক', MACROLIDE: 'ম্যাক্রোলাইড অ্যান্টিবায়োটিক', TRICYC:
'ট্রাইসাইক্লিক অবসাদের ওষুধ' },
    colIngredient: 'মূল উপাদান', usedAs: 'যে নামে পেয়েছেন', fix: 'আপনি কি বোঝাতে চেয়েছেন', unknownT: 'চেনা যায়নি — জেনেরিক নাম লিখুন বা
বানান দেখুন:',
    dq: 'ডাক্তারকে যে প্রশ্নগুলো করবেন',
    q: {
      SAME_GENERIC: (l) => `${l} — সবগুলো কি আমার লাগবে, নাকি একটি?`,
      OVERLAP: (l) => `${l} একটির ভেতরে আরেকটি আছে — কোনটি রাখব?`,
      CLASS_DUP: (l) => `${l} একসাথে খাওয়ার কথা কি আপনি ভেবেছেন?`,
      SAME_BRAND: (l) => `${l} দুটি প্রেসক্রিপশনে আছে — একটি কি পুনরাবৃত্তি?`,
      INTERACTION: (a, b) => `${a} ও ${b} একসাথে খাওয়া কি নিরাপদ?`,
    },
    scanBusy: 'প্রেসক্রিপশন পড়া হচ্ছে…', scanNoKey: 'ছবি স্ক্যানে “স্ক্যান সেটিংস”-এ Gemini key দিন — অথবা নাম টাইপ করুন / নমুনা ব্যবহার করুন।',
    scanDone: (n) => `ছবি থেকে ${n}টি ওষুধ পড়া হয়েছে। নিচে মিলিয়ে দেখুন।`, scanFail: 'ছবিটি পড়া যায়নি। আরও স্পষ্ট ছবি দিন বা নাম টাইপ করুন।',
    loading: 'বাংলাদেশের ওষুধ তালিকা লোড হচ্ছে…',
    sampleTag: 'নমুনা', scanned: 'স্ক্যান করা প্রেসক্রিপশন',
  },
};

let lang = localStorage.getItem('mm-lang') || 'en';
let index = null, samples = [];
let rxs = []; // [{doctor, text}]
const t = () => T[lang];

function applyLang() {
  document.documentElement.lang = lang;
  document.body.classList.toggle('bn', lang === 'bn');
  document.querySelectorAll('[data-i18n]').forEach((el) => { const v = t()[el.dataset.i18n]; if (typeof v ===
'string') el.innerHTML = v; });
  document.querySelectorAll('.lang button').forEach((b) => b.classList.toggle('on', b.dataset.lang === lang));
  renderSamples(); renderLane(); update();
}

function renderSamples() {
  $('#samples').innerHTML = samples.map((s, i) => `<button class="sample" data-i="${i}"><i>${t().sampleTag}</i
><b>${esc(s.title[lang])}</b><span>${esc(s.note[lang])}</span></button>`).join('');
  document.querySelectorAll('.sample').forEach((b) => b.addEventListener('click', () => {
    const s = samples[+b.dataset.i];
    rxs = s.prescriptions.map((p) => ({ doctor: `${p.doctor} · ${p.date}`, text: p.items.join('\n') }));
    renderLane(); update(); $('#bench').scrollIntoView({ behavior: 'smooth', block: 'start' });
  }));
}

function lineChips(text) {
  const lines = text.split('\n').map((l) => l.trim()).filter(Boolean);
  return lines.map((l) => {
    const r = resolveItem(index, l);
    if (!r.kind) return `<li><span class="tag no">?</span> <span>${esc(l)}</span></li>`;
    const cls = r.status === 'fuzzy' ? 'fz' : 'ok';
    const label = r.kind === 'brand' ? `${esc(r.display)} <span class="arrow">→</span> ${esc(r.generic)}` : es
c(r.generic);
    return `<li><span class="tag ${cls}">${r.status === 'fuzzy' ? '≈' : '✓'}</span> <span>${label}</span></li>
`;
  }).join('');
}

function renderLane() {
  if (!index) return;
  const lane = $('#lane');
  lane.innerHTML = rxs.map((r, i) => `
    <article class="rx" data-i="${i}">
      <button class="rm" title="${t().remove}" aria-label="${t().remove}">×</button>
      <header><input class="doc" value="${esc(r.doctor)}" placeholder="${t().doctor}" aria-label="${t().docto
r}"><span class="rxn">${t().rx} ${i + 1}</span></header>
```

```
      <textarea spellcheck="false" aria-label="Medicines">${esc(r.text)}</textarea>
      <ul class="lines">${lineChips(r.text)}</ul>
    </article>`).join('');
  lane.querySelectorAll('.rx').forEach((el) => {
    const i = +el.dataset.i;
    el.querySelector('.rm').onclick = () => { rxs.splice(i, 1); renderLane(); update(); };
    el.querySelector('.doc').oninput = (e) => { rxs[i].doctor = e.target.value; update(); };
    let timer;
    el.querySelector('textarea').oninput = (e) => {
      rxs[i].text = e.target.value;
      el.querySelector('.lines').innerHTML = lineChips(rxs[i].text);
      clearTimeout(timer); timer = setTimeout(update, 250);
    };
  });
}

function update() {
  if (!index) return;
  const prescriptions = rxs.map((r) => ({ items: r.text.split('\n').map((l) => l.trim()).filter(Boolean) }));
  const total = prescriptions.reduce((n, p) => n + p.items.length, 0);
  const res = $('#result');
  if (total < 2) { res.hidden = true; return; }
  const out = reconcile(index, prescriptions);
  res.hidden = false;
  const nHigh = out.counts.high, nMed = out.counts.medium;
  const [big, head, body] = nHigh ? t().verdictHigh(nHigh) : nMed ? t().verdictMed(nMed) : t().verdictOk();
  const more = nHigh && nMed ? ` ${t().more(nMed)}` : '';
  $('#verdict').className = 'verdict ' + out.level;
  $('#verdict').innerHTML = `<div class="big">${esc(big)}</div><div><h3>${esc(head)}</h3><p>${esc(body + mor
e)}</p><p class="small">${out.counts.medicines} ${t().meds} · ${out.counts.distinct} ${t().distinct}</p></div>
`;

  const names = (ids) => [...new Set(ids.map((i) => out.items[i].display))];
  const plain = (arr) => arr.length < 2 ? arr.join('') : arr.slice(0, -1).join(', ') + t().and + arr[arr.lengt
h - 1];
  const bold = (arr) => plain(arr.map((x) => `<b>${esc(x)}</b>`));
  const chips = (arr) => arr.map((x) => `<span class="chip">${esc(x)}</span>`).join('');
  const sideNames = (f) => f.sides.map(names);
  $('#findings').innerHTML = out.findings.length ? out.findings.map((f, k) => {
    const [title, tpl] = t()[f.type];
    const all = names(f.items);
    const sev = f.type === 'INTERACTION' ? (f.severity === 'severe' ? t().sevSerious : t().sevCaution) : (f.le
vel === 'high' ? t().checkNow : t().askDoc);
    const snippet = f.snippet ? `<q>${t().source} ${esc(f.snippet)}</q>` : '';
    let body;
    if (f.type === 'INTERACTION') { const [sa, sb] = sideNames(f); body = tpl(bold(sa), bold(sb)); }
    else { const g = f.type === 'CLASS_DUP' ? esc(t().groups[f.group] || f.group) : esc(f.generic || ''); body
= tpl(bold(all), g); }
    return `<article class="f ${f.level}" style="animation-delay:${k * 40}ms"><div class="t"><span class="pill
${f.level === 'high' ? '' : 'm'}">${esc(sev)}</span><b>${esc(title)}</b></div>
      <div class="chips">${chips(all)}</div><p>${body}</p>${snippet}</article>`;
  }).join('') : '';

  $('#merged').innerHTML = out.merged.map((r) => `<div class="m-row ${r.flagged ? 'flag' : ''}"><b>${esc(r.gen
eric)}</b><small>${r.brands.reduce((s, b) => s + b.rx.length, 0)}×</small>
    <div class="brands">${r.brands.map((b) => `<span>${esc(b.name)}${b.strength ? ' ' + esc(b.strength) : ''}
</span>`).join('')}</div></div>`).join('');

  $('#unknown').innerHTML = out.unknown.length ? `<div class="unk">${t().unknownT} ${out.unknown.map((u) => `<
b>${esc(u.raw)}</b>`).join(', ')}</div>` : '';

  const qs = [];
  for (const f of out.findings.slice(0, 5)) {
    const fn = t().q[f.type]; if (!fn) continue;
    const q = f.type === 'INTERACTION' ? fn(plain(names(f.sides[0])), plain(names(f.sides[1]))) : fn(plain(nam
es(f.items)));
    qs.push(`<li>${esc(q)}</li>`);
  }
  $('#docq').innerHTML = qs.length ? `<h4>${t().dq}</h4><ul>${qs.join('')}</ul>` : '';
```

```
  $('#docq').hidden = !qs.length;
}

// ---- photo scan (Gemini, from the browser, using the user's own key) ----
async function downscale(file, max = 1600) {
  const bmp = await createImageBitmap(file);
  const k = Math.min(1, max / Math.max(bmp.width, bmp.height));
  const c = document.createElement('canvas'); c.width = Math.round(bmp.width * k); c.height = Math.round(bmp.h
eight * k);
  c.getContext('2d').drawImage(bmp, 0, 0, c.width, c.height);
  return c.toDataURL('image/jpeg', 0.85).split(',')[1];
}
async function scanPhoto(file) {
  const key = localStorage.getItem('mm-key');
  const st = $('#scanStatus');
  if (!key) { st.textContent = t().scanNoKey; $('#keyDlg').showModal(); return; }
  st.textContent = t().scanBusy;
  try {
    const b64 = await downscale(file);
    const prompt = 'You read handwritten or printed Bangladeshi medical prescriptions. Extract every medicine.
Return JSON only: {"doctor": string, "date": string, "medicines": [{"name": string (brand or generic as writte
n), "strength": string, "frequency": string}]}. Use "" for unknown fields. Do not invent medicines; if a word
is illegible, skip it.';
    const r = await fetch(`https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateCo
ntent?key=${encodeURIComponent(key)}`, {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ contents: [{ parts: [{ text: prompt }, { inline_data: { mime_type: 'image/jpeg',
data: b64 } }] }], generationConfig: { responseMimeType: 'application/json', temperature: 0 } }),
    });
    if (!r.ok) throw new Error('HTTP ' + r.status);
    const j = await r.json();
    const data = JSON.parse(j.candidates[0].content.parts[0].text);
    const meds = (data.medicines || []).filter((m) => m && m.name);
    if (!meds.length) throw new Error('empty');
    rxs.push({ doctor: [data.doctor, data.date].filter(Boolean).join(' · ') || t().scanned, text: meds.map((m)
=> [m.name, m.strength, m.frequency].filter(Boolean).join(' ')).join('\n') });
    renderLane(); update(); st.textContent = t().scanDone(meds.length);
  } catch (e) { st.textContent = t().scanFail; }
}

// ---- wiring ----
document.querySelectorAll('.lang button').forEach((b) => b.addEventListener('click', () => { lang = b.dataset.
lang; localStorage.setItem('mm-lang', lang); applyLang(); }));
$('#addRx').onclick = () => { rxs.push({ doctor: '', text: '' }); renderLane(); update(); $('#lane').lastEleme
ntChild?.querySelector('textarea').focus(); };
$('#clearAll').onclick = () => { rxs = [{ doctor: '', text: '' }, { doctor: '', text: '' }]; renderLane(); upd
ate(); };
$('#photo').onchange = (e) => { if (e.target.files[0]) scanPhoto(e.target.files[0]); e.target.value = ''; };
$('#keyBtn').onclick = () => { $('#keyInput').value = localStorage.getItem('mm-key') || ''; $('#keyDlg').showM
odal(); };
$('#saveKey').onclick = () => { const v = $('#keyInput').value.trim(); if (v) localStorage.setItem('mm-key',
v); else localStorage.removeItem('mm-key'); };
$('#print').onclick = () => window.print();

(async function init() {
  $('#scanStatus').textContent = t().loading;
  const [data, s] = await Promise.all([fetch('data/medmerge-data.json').then((r) => r.json()), fetch('data/sam
ples.json').then((r) => r.json())]);
  index = buildIndex(data); samples = s.samples;
  $('#credit').innerHTML = `Brand and label data: <a href="https://github.com/wsayan/medicinedb">wsayan/medici
nedb</a> (MIT) · ${Object.keys(data.brands).length.toLocaleString()} brands · ${Object.keys(data.generics).len
gth.toLocaleString()} generics. Interaction wording comes from those label summaries and has not been clinical
ly validated.`;
  $('#scanStatus').textContent = '';
  rxs = [{ doctor: '', text: '' }, { doctor: '', text: '' }];
  applyLang();
  if (new URLSearchParams(location.search).get('demo') === '1') document.querySelector('.sample')?.click();
})();
```

engine.js

```
// MedMerge reconciliation engine — pure functions, no DOM. Runs in the browser and in Node (tests).
// Data comes from data/medmerge-data.json (Bangladeshi brand -> generic index, see scripts/build_data.py).

const FORM_WORDS = /\b(tab|tabs|tablet|tablets|cap|caps|capsule|capsules|syp|syrup|susp|suspension|inj|injecti
on|drops?|cream|oint(ment)?|gel|sachet|supp(ository)?)\b\.?/gi;
const NOISE = new Set(['days', 'day', 'after', 'before', 'meal', 'meals', 'daily', 'bd', 'tds', 'od', 'qid',
'hs', 'sos', 'x', 'for', 'mg', 'ml', 'tablet', 'tab']);
const TOPICAL = /(\beye\b|\bear\b|nasal|cream|ointment|gel|lotion|shampoo|spray|inhal|drops|paint|powder for|v
aginal|enema|solution)/i;

// Therapeutic-class groups used for "same class" and interaction wording
export const GROUP_ALIASES = {
  NSAID: /nsaids?|non-?steroidal/i, ANTICOAG: /anticoagulants?|coumarins?/i, ANTIPLT: /antiplatelet|anti-plate
let/i,
  ACEI: /ace inhibitors?/i, ARB: /angiotensin/i, BETA: /beta[- ]blockers?/i, CCB: /calcium[- ]channel/i,
  SSRI: /ssris?|selective serotonin/i, BZD: /benzodiazepines?/i, OPIOID: /opioids?/i, CORTICO: /corticosteroid
s?/i,
  LOOPDIU: /diuretics?/i, THIAZ: /diuretics?/i, SU: /sulf?onylureas?|hypoglyc[a]?emics?/i, PPI: /proton pump/
i,
  H2: /h2[- ]antagonists?/i, STATIN: /statins?/i, MACROLIDE: /macrolides?/i, FLUOROQ: /quinolones?/i,
  TRICYC: /tricyclic/i, ANTIHIST: /antihistamines?/i,
};
// Class wording is only trusted for these (broad words like "diuretics" or "antiplatelet" are too loose)
const CLASS_MATCH = Object.fromEntries(['NSAID', 'ANTICOAG', 'ACEI', 'ARB', 'BETA', 'CCB', 'SSRI', 'BZD', 'OPI
OID', 'SU', 'STATIN', 'MACROLIDE', 'FLUOROQ', 'TRICYC'].map((k) => [k, GROUP_ALIASES[k]]));
// Classes where taking two of them together is normal care (not flagged as a duplicate class)
const CLASS_DUP_EXEMPT = new Set(['ANTIPLT', 'CORTICO', 'CEPH', 'PENICILLIN', 'BETA', 'LOOPDIU', 'THIAZ', 'ANT
ICOAG']);

export function lev(a, b, max = 3) {
  if (Math.abs(a.length - b.length) > max) return max + 1;
  let prev = Array.from({ length: b.length + 1 }, (_, i) => i);
  for (let i = 1; i <= a.length; i++) {
    const cur = [i];
    let rowMin = i;
    for (let j = 1; j <= b.length; j++) {
      cur[j] = Math.min(prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + (a[i - 1] === b[j - 1] ? 0 : 1));
      rowMin = Math.min(rowMin, cur[j]);
    }
    if (rowMin > max) return max + 1;
    prev = cur;
  }
  return prev[b.length];
}

export function buildIndex(data) {
  const brandKeys = Object.keys(data.brands);
  const byFirst = new Map();
  for (const k of brandKeys) {
    const c = k[0];
    if (!byFirst.has(c)) byFirst.set(c, []);
    byFirst.get(c).push(k);
  }
  const genericByName = new Map(); // lowercase generic full name / key -> [gid]
  const add = (name, gid) => {
    if (!name) return;
    if (!genericByName.has(name)) genericByName.set(name, []);
    const l = genericByName.get(name);
    if (!l.includes(gid)) l.push(gid);
  };
  for (const [gid, g] of Object.entries(data.generics)) {
    add(g.n.toLowerCase(), gid);
    if (g.k.length === 1) add(g.k[0], gid);
  }
  const drugWords = new Set();
  for (const g of Object.values(data.generics)) for (const k of g.k) { const w = k.split(' ')[0]; if (w.length
>= 6) drugWords.add(w); }
  return { data, brandKeys, byFirst, genericByName, drugWords };
```

```
}

export function parseLine(input) {
  if (typeof input !== 'string') {
    return { raw: input.name || '', name: (input.name || '').trim(), strength: input.strength || '', freq: inp
ut.frequency || input.freq || '', form: input.form || '' };
  }
  let s = input.replace(/\([^)]*\)/g, ' ');
  const f = s.match(/\b\d\s*[+\-]\s*\d\s*[+\-]\s*\d(?:\s*[+\-]\s*\d)?\b/);
  const freq = f ? f[0].replace(/\s+/g, '') : '';
  if (f) s = s.replace(f[0], ' ');
  const st = s.match(/(\d+(?:\.\d+)?)\s*(mg|mcg|µg|gm|g|ml|iu|%)/i);
  const strength = st ? st[0].replace(/\s+/g, '').toLowerCase() : '';
  if (st) s = s.replace(st[0], ' ');
  const formM = s.match(FORM_WORDS);
  s = s.replace(FORM_WORDS, ' ');
  const toks = s.toLowerCase().replace(/[^a-z0-9\-+ ]/g, ' ').split(/\s+/)
    .filter((t) => t && !NOISE.has(t) && !/^\d+$/.test(t));
  return { raw: input.trim(), name: toks.join(' '), strength, freq, form: formM ? formM[0].toLowerCase() : ''
};
}

function strengthNums(s) { return (s || '').toLowerCase().match(/\d+(?:\.\d+)?/g) || []; }

function pickEntries(entries, strength, form) {
  // prefer oral solids (what patients actually take); then narrow by strength / form if the label offers them
  const oral = entries.filter((e) => /tablet|capsule/i.test(e[1]));
  let pool = oral.length ? oral : entries;
  if (strength) {
    const want = strengthNums(strength)[0];
    const hit = pool.filter((e) => strengthNums(e[2]).includes(want));
    if (hit.length) pool = hit;
  }
  if (form) {
    const hit = pool.filter((e) => e[1].toLowerCase().includes(form.replace(/s$/, '')));
    if (hit.length) pool = hit;
  }
  return pool;
}

export function resolveItem(index, input) {
  const p = parseLine(input);
  const { data } = index;
  const base = { raw: p.raw, strength: p.strength, freq: p.freq, query: p.name };
  if (!p.name) return { ...base, status: 'unknown', display: p.raw };
  const toks = p.name.split(' ');
  // 1) exact brand, longest prefix first
  for (let n = toks.length; n >= 1; n--) {
    const key = toks.slice(0, n).join(' ');
    if (data.brands[key]) return fromBrand(index, base, key, p, 'exact');
  }
  // 2) exact generic name
  for (let n = toks.length; n >= 1; n--) {
    const key = toks.slice(0, n).join(' ');
    const g = index.genericByName.get(key);
    if (g) return fromGeneric(index, base, g, p);
  }
  // 3) fuzzy brand (OCR / typing slips), then fuzzy generic
  const key = toks.slice(0, Math.min(2, toks.length)).join(' ');
  const cand = [];
  const pool = (index.byFirst.get(key[0]) || []).concat(index.byFirst.get(toks[0][0]) || []);
  const max = key.length <= 5 ? 1 : 2;
  for (const k of new Set(pool)) {
    const d = Math.min(lev(key, k, max), lev(toks[0], k, max));
    if (d <= max) cand.push([d, k]);
  }
  cand.sort((a, b) => a[0] - b[0] || a[1].length - b[1].length);
  if (cand.length) {
    const r = fromBrand(index, base, cand[0][1], p, 'fuzzy');
```

```
    r.alternatives = cand.slice(1, 4).map((c) => c[1]);
    return r;
  }
  for (const [name, g] of index.genericByName) {
    if (lev(toks[0], name.split(' ')[0], max) <= max) return { ...fromGeneric(index, base, g, p), status: 'fuz
zy' };
  }
  return { ...base, status: 'unknown', display: p.raw };
}

function fromBrand(index, base, key, p, status) {
  const { data } = index;
  const all = data.brands[key];
  const pool = pickEntries(all, p.strength, p.form);
  const gids = [...new Set(pool.map((e) => e[0]))];
  const entry = pool[0];
  const gid = entry[0];
  return {
    ...base, status, kind: 'brand', display: titleCase(key), brand: key,
    gid, gids, ambiguous: gids.length > 1,
    form: entry[1], pickedStrength: entry[2], company: entry[3],
    strengthsAvailable: [...new Set(all.map((e) => e[2]).filter(Boolean))].slice(0, 6),
    generic: shortGeneric(data.generics[gid].n), keys: data.generics[gid].k, groups: data.generics[gid].g,
    text: data.generics[gid].i, systemic: !TOPICAL.test(entry[1] + ' ' + data.generics[gid].n),
  };
}

function fromGeneric(index, base, gids, p) {
  const { data } = index;
  const gid = gids.find((id) => data.generics[id].k.length === 1) || gids[0];
  const g = data.generics[gid];
  return {
    ...base, status: 'exact', kind: 'generic', display: shortGeneric(g.n), brand: null, gid, gids, ambiguous:
false,
    form: '', pickedStrength: '', company: '', strengthsAvailable: [],
    generic: shortGeneric(g.n), keys: g.k, groups: g.g, text: g.i, systemic: !TOPICAL.test(g.n),
  };
}

const SALT_TAIL = /\b(trihydrate|dihydrate|monohydrate|hemihydrate|anhydrous|bp|usp|ip|hydrochloride|hcl|magne
sium|sodium|potassium|maleate|besilate|besylate|tartrate|succinate|fumarate|sulfate|sulphate|bisulfate|bisulph
ate|dipropionate|mesylate)\b/gi;
export function shortGeneric(n) { const r = n.replace(SALT_TAIL, ' ').replace(/\s+/g, ' ').replace(/\s*\+\s*/
g, ' + ').trim(); return r || n; }
export function titleCase(s) { return s.replace(/\b[a-z]/g, (c) => c.toUpperCase()); }

const sigOf = (it) => [...it.keys].sort().join('|');

function sentenceFor(text, re) {
  // Label text keeps a separate "Potentially Fatal:" line; only a hit on that line counts as severe.
  const lines = text.split(/\n+/);
  for (const line of lines) {
    if (!re.test(line)) continue;
    const fatal = /^\s*potentially fatal/i.test(line);
    const pieces = line.replace(/^\s*potentially fatal:?\s*/i, '').split(/(?<=[.;])\s+/);
    const hit = pieces.find((p) => re.test(p)) || line;
    const clean = hit.trim();
    return { sev: fatal ? 'severe' : 'caution', snippet: clean.length > 260 ? clean.slice(0, 257) + '…' : clea
n };
  }
  return null;
}

function interactionBetween(index, a, b) {
  // does a's label text mention b? (by ingredient name, else by class wording)
  const t = a.text || '';
  if (!t) return null;
  for (const k of b.keys) {
    const first = k.split(' ')[0];
```

```
    if (first.length < 5) continue;
    const re = new RegExp('\\b' + first.replace(/[-/\\^$*+?.()|[\]{}]/g, '\\$&'), 'i');
    const r = sentenceFor(t, re);
    if (r) return { ...r, via: 'ingredient' };
  }
  for (const g of b.groups) {
    if (a.groups.includes(g)) continue; // same-class pairs are reported as class duplicates
    const re = CLASS_MATCH[g];
    if (!re) continue;
    const r = sentenceFor(t, new RegExp(re.source, 'i'));
    if (!r) continue;
    // a class wording inside a sentence about one specific other drug ("antiplatelet effects of clopidogrel")
is not about b
    const bWords = new Set(b.keys.map((k) => k.split(' ')[0]));
    const named = (r.snippet.toLowerCase().match(/[a-z]{6,}/g) || []).filter((w) => index.drugWords.has(w) &&
!bWords.has(w));
    if (named.length) continue;
    return { ...r, via: 'class', group: g };
  }
  return null;
}

export function reconcile(index, prescriptions) {
  const items = [];
  prescriptions.forEach((rx, ri) => {
    (rx.items || []).forEach((line) => {
      const r = resolveItem(index, line);
      items.push({ ...r, rx: ri, idx: items.length });
    });
  });
  const known = items.filter((i) => i.kind && i.systemic);
  const findings = [];
  const seen = new Set();
  const push = (f) => { const k = f.type + ':' + f.items.slice().sort().join(','); if (!seen.has(k)) { seen.ad
d(k); findings.push({ ...f, sa: sigOf(items[f.items[0]]), sb: sigOf(items[f.items[1]]) }); } };

  for (let x = 0; x < known.length; x++) {
    for (let y = x + 1; y < known.length; y++) {
      const A = known[x], B = known[y];
      const sa = sigOf(A), sb = sigOf(B);
      if (sa === sb) {
        if (A.brand && A.brand === B.brand) {
          if (A.rx !== B.rx) push({ type: 'SAME_BRAND', level: 'medium', items: [A.idx, B.idx], generic: A.gen
eric });
        } else {
          push({ type: 'SAME_GENERIC', level: 'high', items: [A.idx, B.idx], generic: A.generic });
        }
        continue;
      }
      const shared = A.keys.filter((k) => B.keys.includes(k));
      if (shared.length) {
        push({ type: 'OVERLAP', level: 'high', items: [A.idx, B.idx], generic: shared.join(', ') });
        continue;
      }
      const g = A.groups.find((gr) => B.groups.includes(gr) && !CLASS_DUP_EXEMPT.has(gr));
      if (g) { push({ type: 'CLASS_DUP', level: 'medium', items: [A.idx, B.idx], group: g }); continue; }
      const ab = interactionBetween(index, A, B), ba = interactionBetween(index, B, A);
      const hit = [ab, ba].filter(Boolean).sort((p, q) => (p.sev === 'severe' ? -1 : 1) - (q.sev === 'severe'
? -1 : 1))[0];
      if (hit) push({ type: 'INTERACTION', level: hit.sev === 'severe' ? 'high' : 'medium', items: [A.idx, B.i
dx], snippet: hit.snippet, severity: hit.sev });
    }
  }
  // one card per problem, not one per pair: Seclo + Losectil + Maxpro are listed together
  const grouped = new Map();
  for (const f of findings) {
    const sigs = [f.sa, f.sb].sort();
    const key = f.type === 'SAME_GENERIC' ? 'SG:' + f.sa : f.type === 'SAME_BRAND' ? 'SB:' + items[f.items
[0]].brand
```

```
      : f.type === 'CLASS_DUP' ? 'CD:' + f.group : f.type + ':' + sigs.join('#');
    const g = grouped.get(key);
    if (!g) { grouped.set(key, { ...f, sigs, items: [...f.items] }); continue; }
    for (const i of f.items) if (!g.items.includes(i)) g.items.push(i);
    if (f.severity === 'severe' && g.severity !== 'severe') { g.severity = 'severe'; g.level = 'high'; g.snipp
et = f.snippet; }
  }
  findings.length = 0;
  for (const g of grouped.values()) {
    g.items.sort((x, y) => x - y);
    if (g.type === 'INTERACTION' || g.type === 'OVERLAP') {
      g.sides = [g.sigs[0], g.sigs[1]].map((sg) => g.items.filter((i) => sigOf(items[i]) === sg));
      if (g.type === 'INTERACTION') { const iA = items[g.sides[0][0]]; g.sidesGeneric = g.sides.map((sd) => it
ems[sd[0]].generic); }
    }
    findings.push(g);
  }
  const rank = { high: 0, medium: 1, info: 2 };
  findings.sort((p, q) => rank[p.level] - rank[q.level]);

  // merged list: one row per distinct active ingredient set
  const rows = new Map();
  for (const it of items) {
    if (!it.kind) continue;
    const sig = sigOf(it);
    if (!rows.has(sig)) rows.set(sig, { sig, generic: it.generic, groups: it.groups, brands: [], flagged: fals
e });
    const row = rows.get(sig);
    let b = row.brands.find((x) => x.name === it.display);
    if (!b) { b = { name: it.display, rx: [], strength: it.pickedStrength || it.strength, freq: it.freq }; ro
w.brands.push(b); }
    if (!b.rx.includes(it.rx)) b.rx.push(it.rx);
  }
  const bad = new Set();
  for (const f of findings) for (const i of f.items) bad.add(sigOf(items[i]));
  for (const r of rows.values()) r.flagged = bad.has(r.sig);
  const unknown = items.filter((i) => !i.kind);
  const level = findings.some((f) => f.level === 'high') ? 'high' : findings.length ? 'medium' : 'ok';
  return {
    items, findings, merged: [...rows.values()], unknown, level,
    counts: { medicines: items.length, distinct: rows.size, high: findings.filter((f) => f.level === 'high').l
ength, medium: findings.filter((f) => f.level === 'medium').length },
  };
}
```

scripts/build_data.py

```
#!/usr/bin/env python3
"""Build data/medmerge-data.json from the Bangladeshi medicine SQLite DB.

Source: github.com/wsayan/medicinedb (MIT, (c) 2021 Wahid Sadique Ayan), itself compiled from
public Bangladeshi drug-index data. Run:  python3 scripts/build_data.py path/to/medicine.db
"""
import json, re, sqlite3, sys

db = sqlite3.connect(sys.argv[1] if len(sys.argv) > 1 else "medicine.db")

SALTS = set("""hydrochloride hcl sodium potassium calcium magnesium trihydrate dihydrate monohydrate hemihydra
te
anhydrous bp usp ip maleate besilate besylate mesylate tartrate succinate fumarate sulfate sulphate phosphate
acetate
citrate bromide chloride nitrate oxalate lactate gluconate hydrobromide dipropionate propionate valerate bisul
fate
bisulphate hydrogen disodium dipotassium pentahydrate sesquihydrate heptahydrate decahydrate micronized micron
ised
film coated enteric and of as""".split())

def keys_of(name):
    out = []
    for part in name.split("+"):
```

```
        part = re.sub(r"\(.*?\)", " ", part.lower())
        part = re.sub(r"[\d.%/]+\s*(mg|mcg|gm|g|ml|iu|%)?", " ", part)
        toks = [t for t in re.findall(r"[a-z][a-z\-]*", part) if t not in SALTS]
        if toks:
            out.append(" ".join(toks[:3]))
    return sorted(set(out))

GROUPS = [  # (group id, regex on therapeutic class name)
    ("NSAID", r"NSAID"), ("PPI", r"proton pump"), ("H2", r"H2 (receptor )?antagonist"),
    ("STATIN", r"^Statins"), ("ACEI", r"ACE\)"), ("ARB", r"Angiotensin-II receptor"),
    ("BETA", r"[Bb]eta[- ](adreno|blockers)|Beta-blockers|Beta blockers"),
    ("CCB", r"Calcium[- ]channel"), ("SSRI", r"SSRI"), ("BZD", r"Benzodiazepine (hypnotics|sedatives)"),
    ("OPIOID", r"^Opioid analgesics"), ("SU", r"Sulfonylureas"), ("ANTICOAG", r"anti-?coagulants"),
    ("ANTIPLT", r"[Aa]nti-platelet"), ("CORTICO", r"^Corticosteroid|^Glucocorticoids"),
    ("LOOPDIU", r"Loop diuretics"), ("THIAZ", r"Thiazide"), ("ANTIHIST", r"[Aa]nti-?histamine"),
    ("FLUOROQ", r"4-Quinolone"), ("MACROLIDE", r"Macrolides"), ("CEPH", r"Cephalosporins"),
    ("PENICILLIN", r"penicillin"), ("TRICYC", r"Tricyclic"),
]
cls_of = {}
for gid, cname in db.execute("select tg.generic_id, t.therapitic_name from therapitic_generic tg join therapit
ic t on t.therapitic_id=tg.therapitic_id"):
    for g, rx in GROUPS:
        if re.search(rx, cname, re.I):
            cls_of.setdefault(gid, set()).add(g)

generics = {}
for gid, name, inter in db.execute("select generic_id, generic_name, interaction from generic"):
    generics[gid] = {
        "n": name.strip(),
        "k": keys_of(name),
        "g": sorted(cls_of.get(gid, [])),
        "i": (inter or "").strip(),
    }
companies = {cid: n for cid, n, _ in db.execute("select company_id, company_name, company_order from company_n
ame")}
brands = {}
for bid, gid, cid, bname, form, strength, price, pack in db.execute(
        "select brand_id, generic_id, company_id, brand_name, form, strength, price, packsize from brand"):
    if gid not in generics or not bname:
        continue
    key = re.sub(r"\s+", " ", bname.strip().lower())
    entry = [gid, (form or "").strip(), (strength or "").strip(), companies.get(cid, "")]
    lst = brands.setdefault(key, [])
    if entry not in lst:
        lst.append(entry)

# generics filed under "Non-opioid analgesics" that are NSAIDs
for gid, g in generics.items():
    if "ketorolac" in g["k"] and "tromethamine" in g["n"].lower() and "%" not in g["n"]:
        g["g"] = sorted(set(g["g"]) - {"OPIOID"} | {"NSAID"})

out = {"generics": generics, "brands": brands,
       "meta": {"source": "wsayan/medicinedb (MIT)", "brands": len(brands), "generics": len(generics)}}
with open("data/medmerge-data.json", "w", encoding="utf-8") as f:
    json.dump(out, f, ensure_ascii=False, separators=(",", ":"))
print(out["meta"])
```

tests/engine.test.mjs

```
import { readFileSync } from 'node:fs';
import assert from 'node:assert/strict';
import { buildIndex, resolveItem, reconcile, parseLine } from '../engine.js';

const data = JSON.parse(readFileSync(new URL('../data/medmerge-data.json', import.meta.url)));
const samples = JSON.parse(readFileSync(new URL('../data/samples.json', import.meta.url))).samples;
const idx = buildIndex(data);
const has = (res, type, a, b) => res.findings.some((f) => f.type === type &&
  (!a || (f.items.some((i) => res.items[i].display === a) && f.items.some((i) => res.items[i].display ===
b))));
```

```
// parsing
assert.equal(parseLine('Tab. Napa 500 mg 1+1+1').name, 'napa');
assert.equal(parseLine('Tab. Napa 500 mg 1+1+1').freq, '1+1+1');
assert.equal(parseLine('Cap. Seclo 20mg').strength, '20mg');

// resolution
assert.equal(resolveItem(idx, 'Napa').generic, 'Paracetamol');
assert.equal(resolveItem(idx, 'Seclo 20 mg').generic, 'Omeprazole');
assert.equal(resolveItem(idx, 'Napaa').status, 'fuzzy');          // typo / OCR slip
assert.equal(resolveItem(idx, 'warfarin').kind, 'generic');       // generic names work too
assert.equal(resolveItem(idx, 'zzzqqq').status, 'unknown');

// scenario 1: Rahima
const r1 = reconcile(idx, samples[0].prescriptions);
console.log('rahima', r1.counts, r1.findings.map((f) => `${f.type}/${f.level}: ${f.items.map((i) => r1.items
[i].display).join(' + ')}`));
assert.ok(has(r1, 'SAME_GENERIC', 'Napa', 'Ace'), 'Napa vs Ace = same paracetamol');
assert.ok(has(r1, 'SAME_GENERIC', 'Seclo', 'Losectil'), 'Seclo vs Losectil = same omeprazole');
assert.ok(has(r1, 'OVERLAP', 'Napa', 'Napa Extra'), 'Napa Extra hides paracetamol');
assert.ok(has(r1, 'CLASS_DUP', 'Maxpro', 'Seclo'), 'two PPIs');
assert.equal(r1.level, 'high');

// scenario 2: Karim — two NSAIDs
const r2 = reconcile(idx, samples[1].prescriptions);
console.log('karim', r2.counts, r2.findings.map((f) => `${f.type}/${f.level}: ${f.items.map((i) => r2.items
[i].display).join(' + ')}`));
assert.ok(has(r2, 'CLASS_DUP', 'Tory', 'Voltalin'), 'two NSAIDs');

// scenario 3: clean
const r3 = reconcile(idx, samples[2].prescriptions);
console.log('clean', r3.counts, r3.findings.map((f) => `${f.type}/${f.level}`));
assert.equal(r3.level, 'ok');
console.log('ALL OK');
for (const f of r1.findings.filter((f) => f.type === 'INTERACTION')) console.log(r1.items[f.items[0]].display,
'+', r1.items[f.items[1]].display, f.severity, '|', f.snippet);
```

README.md

```
# MedMerge — one safe medicine list from every prescription

**Hackathon:** DSH Hacks V2 · **Theme:** AI × Healthcare · **Live demo:** https://shahriarsharif406.github.io/
medmerge/

In Bangladesh one medicine is sold under dozens of brand names. Napa and Ace are both paracetamol; Seclo and
Losectil are both omeprazole. When a patient collects prescriptions from several doctors, a double dose can hi
de
behind two different names. MedMerge reads the prescriptions, resolves every brand to its active ingredient,
merges them into one list, and shows what needs a second look — in English and Bangla.

## What it flags
| Finding | Example |
|---|---|
| Same medicine, different brand names | Napa + Ace (both paracetamol) |
| Hidden overlap inside a combination | Napa Extra (paracetamol + caffeine) next to Napa |
| Two of the same drug family | Seclo + Maxpro + Losectil (all acid-reducing PPIs) |
| Possible interaction, quoted from the drug label summary | Plavix + Maxpro |

It also writes a short "questions to ask your doctor" list and a printable merged list.

## Try it
Open the live demo and click a sample ("Rahima Begum"), or type medicine names, one per line
(`Tab. Napa 500 mg 1+1+1`). **Scan a photo** reads a prescription image with Google Gemini directly from your
browser using *your own* API key (stored only on your device) — no server, nothing uploaded to us.

## How it works
1. `parseLine` splits a line into name / strength / frequency (Bangladeshi `1+0+1` style supported).
2. `resolveItem` matches the name against 12,000+ Bangladeshi brands, then generics, then fuzzy-matches typos
and OCR slips.
3. `reconcile` compares active ingredients across all prescriptions: duplicates, overlaps, same-class pairs, a
nd label-text interactions.
```

4. `app.js` renders the findings, the merged list and the bilingual UI.

Everything runs in the browser (`engine.js` is pure JavaScript and is unit-tested in Node).

```
node tests/engine.test.mjs        # engine tests
python3 -m http.server            # then open http://localhost:8000
python3 scripts/build_data.py medicine.db   # rebuild data/medmerge-data.json
```

Honest limits
- **Not medical advice.** It never tells anyone to stop a medicine; it tells them what to ask.
- Interaction text comes from label summaries in a 2021 community database and is **not clinically validated**.
  Class-level matching is deliberately conservative; drug–condition, dose and age checks are not implemented.
- Handwriting accuracy depends on Gemini and the photo; every scanned line is shown for the user to confirm.
- Sample patients are synthetic.

Credits
Brand/label data: [wsayan/medicinedb](https://github.com/wsayan/medicinedb) (MIT). See `NOTICE.md`.
Code: MIT, see `LICENSE`. Built by Shahariar / Anvia Labs.